

(SHORT QUESTION AND ANSWERS)

1. Define a) Global Scope b) Local Scope c) Block Scope?

Answer:

a) Global Scope

✔ Definition:

Global Scope means a variable is declared **outside of all functions and blocks**. It can be accessed **anywhere in the program**.

It can be accessed anywhere in the program.

✔ Example:

```
javascript Copy code  
  
let name = "Shaik"; // Global variable  
  
function greet() {  
  console.log(name); // Accessible here  
}  
  
greet();  
console.log(name); // Also accessible here
```

b) Local Scope (Function Scope)

✔ Definition:

Local Scope means a variable is declared **inside a function**. It can be accessed **only inside that function**.

✔ Example:

```
javascript Copy code  
  
function greet() {  
  let message = "Hello"; // Local variable  
  console.log(message);  
}  
  
greet();  
console.log(message); // ❌ Error (not accessible)
```

c) Block Scope

✔ Definition:

Block Scope means a variable is declared inside a **block { }** using **let** or **const**. It can be accessed **only inside that block**.

✓ **Example:**

```
javascript Copy code

if (true) {
  let age = 25; // Block scoped variable
  console.log(age);
}

console.log(age); // ✗ Error
```

2. What is Hoisting?

Answer:

Hoisting is JavaScript's default behavior of **moving declarations to the top of their scope before execution**.

2.1 Function Declarations

Hoisting is a JavaScript mechanism where **function declarations** are moved to the top of their scope before code execution.

```
1 let x = 15;
2 let y = 10;
3 let result = add(x, y);
4 console.log(result); // 25
5
6 function add(a, b) {
7   return a + b;
8 }
```

2.2 Function Expressions

Function expressions in JavaScript are not hoisted.

```
JAVASCRIPT

1 myFunction();
2 let myFunction = function () {
3   let x = 5;
4   console.log(x); // ReferenceError["Cannot access 'myFunction' before initialization"]
5 };
```

2.3 Arrow Functions

Arrow Functions in JavaScript are not hoisted.

```
1 myFunction();
2 let myFunction = () => {
3   let x = 5;
4   console.log(x); // ReferenceError["Cannot access 'myFunction' before initialization"]
5 };
```

3. What is “this” keyword?

Answer:

this keyword in JavaScript refers to the **object that is currently executing the function**.

The **this** is determined in three ways.

- In Object methods, it refers to the object that is executing the current function.
- In Regular functions, it refers to the **window** object.
- In Arrow functions, it refers to the context in which the code is defined.

1.1 `this` in Object Methods

```
1 let car = {
2   color: "blue",
3   brand: "Audi",
4   start: function() {
5     console.log(this); // Object { color: "blue", brand: "Audi", start: f() }
6   }
7 };
8
9 car.start();
```

In the above example, **this** refers to the **car** object as it's executing the method **start** .

1.2 `this` in Regular Functions

```
1 function start() {
2   console.log(this); // Window { }
3 }
4 start();
```

In the above example, `this` refers to the `window` object.

4. What is Object Destructuring?

Answer:

✔ Definition:

Object Destructuring is a JavaScript feature that allows us to **extract properties from an object and store them into variables easily**.

👉 Instead of accessing values using `object.property`, we can directly unpack them into variables.

📌 Quick Syntax

javascript

📄 Copy code

```
const { property1, property2 } = objectName;
```

◆ Normal Way (Without Destructuring)

javascript

📄 Copy code

```
const person = {
  name: "Shaik",
  age: 25,
  city: "Hyderabad"
};

const name = person.name;
const age = person.age;

console.log(name);
console.log(age);
```

◆ Using Object Destructuring

javascript

📄 Copy code

```
const person = {
  name: "Shaik",
  age: 25,
  city: "Hyderabad"
};

const { name, age } = person;

console.log(name);
console.log(age);
```

5. What is Arrow Function?

Answer:

✔ Definition:

An Arrow Function is a shorter and modern way to write functions in JavaScript using the `=>` (arrow) syntax.

👉 Introduced in ES6 (ES2015)

👉 Makes code shorter and cleaner

Syntax:

```
let sum = (param1, param2, ...) => {  
  // statement(s)  
};  
sum();
```

```
1 let sum = (a, b) => {  
2   let result = a + b;  
3   return result;  
4 };  
5 console.log(sum(4, 3));
```

JAVASCRIPT

6. what are the types of Destructuring Syntax?

Answer:

Destructuring in JavaScript is a feature that allows us to **extract values from arrays or objects and assign them to variables easily**.

☐ Array Destructuring

Used to extract values from an **array**.

✔ Syntax:

javascript

Copy code

```
const [var1, var2] = arrayName;
```

✔ Example:

javascript

Copy code

```
const numbers = [10, 20, 30];  
  
const [a, b] = numbers;  
  
console.log(a); // 10  
console.log(b); // 20
```

❖ Object Destructuring

Used to extract values from an **object**.

✅ Syntax: [Ask ChatGPT](#)

javascript

[Copy code](#)

```
const { property1, property2 } = objectName;
```

✅ Example:

javascript

[Copy code](#)

```
const person = {  
  name: "Shaik",  
  age: 25  
};  
  
const { name, age } = person;  
  
console.log(name); // Shaik  
console.log(age);  // 25
```



7. What is synchronous programming in JavaScript?

Answer:

✅ Definition:

Synchronous programming means the code executes **line by line**, one after another.

👉 JavaScript waits for one task to finish before moving to the next task.

◆ Example:

javascript

[Copy code](#)

```
console.log("Step 1");  
console.log("Step 2");  
console.log("Step 3");
```

✅ Output:

vbnet

[Copy code](#)

```
Step 1  
Step 2  
Step 3
```

8. What is asynchronous programming in JavaScript?

Answer:

✔ Definition:

Asynchronous programming in JavaScript allows the program to **start a task and move to the next task without waiting for it to finish.**

👉 It is **non-blocking**.

👉 Used for time-consuming operations like:

- API calls
- File reading
- Database operations
- setTimeout

♦ Example:

javascript

📄 Copy code

```
console.log("Start");

setTimeout(() => {
  console.log("Step 2");
}, 2000);

console.log("End");
```

✔ Output:

powershell

📄 Copy code

```
Start
End
Step 2
```

👉 JavaScript does NOT wait 2 seconds.

It continues executing the next line immediately.



9. What problem does asynchronous JavaScript solve in single threaded execution?

Answer:

Non-blocking: The program continues executing subsequent lines of code while waiting for an asynchronous task to finish.

- **Single-threaded:** JavaScript is naturally single-threaded, meaning it executes one command at a time. It achieves concurrency through its non-blocking behaviour and the event loop.
- **Event Loop:** This mechanism manages the execution of multiple tasks by moving completed asynchronous callbacks from a queue into the call stack once it is empty.
- There are mainly two types in Asynchronous programming:
 - Callback functions
 - Callback Hell and management

10. Define a) Pending state b) Fulfilled state c) Reject state

Answer:

a) Pending State

✓ Definition:

A Promise is in the **Pending** state when it is **still executing** and the result is not yet available.

🔗 Initial state of every Promise.

✓ Example:

```
javascript Copy code  
  
const promise = new Promise((resolve, reject) => {  
  // Task is running  
});
```

At this stage:

- Not resolved
- Not rejected
- Waiting for result

b) Fulfilled State

✓ Definition:

A Promise is in the **Fulfilled** state when the operation is **successful** and `resolve()` is called.

👉 It returns a value.

✅ **Example:**

javascript

📄 Copy code

```
const promise = new Promise((resolve, reject) => {
  resolve("Success");
});

promise.then((result) => {
  console.log(result); // Success
});
```

✔ Task completed successfully

✔ Value is available

c) Rejected State

❌ **Definition:**

A Promise is in the **Rejected** state when the operation **fails** and `reject()` is called.

👉 It returns an error.

✅ **Example:**

javascript

📄 Copy code

```
const promise = new Promise((resolve, reject) => {
  reject("Error occurred");
});

promise.catch((error) => {
  console.log(error); // Error occurred
});
```

✔ Task failed

✔ Error is returned

11. What are the three states of a Promise?

Answer:

a) Pending State

❌ **Definition:**

A Promise is in the **Pending** state when it is **still executing** and the result is not yet available.

👉 Initial state of every Promise.

✅ **Example:**

javascript

📄 Copy code

```
const promise = new Promise((resolve, reject) => {  
  // Task is running  
});
```

At this stage:

- Not resolved
- Not rejected
- Waiting for result

b) Fulfilled State

✅ **Definition:**

A Promise is in the **Fulfilled** state when the operation is **successful** and `resolve()` is called.

👉 It returns a value.

✅ **Example:**

javascript

📄 Copy code

```
const promise = new Promise((resolve, reject) => {  
  resolve("Success");  
});  
  
promise.then((result) => {  
  console.log(result); // Success  
});
```

✔ Task completed successfully

✔ Value is available

c) Rejected State

✅ **Definition:**

A Promise is in the **Rejected** state when the operation **fails** and `reject()` is called.

👉 It returns an error.

✓ Example:

javascript

Copy code

```
const promise = new Promise((resolve, reject) => {  
  reject("Error occurred");  
});  
  
promise.catch((error) => {  
  console.log(error); // Error occurred  
});
```

- ✓ Task failed
- ✓ Error is returned

12. Define .then() and .catch() method in promise?

Answer:

.then(): .then() is used to handle a fulfilled (resolved) Promise. Runs only when resolve() is called.

Syntax: promise.then(successCallback);

.catch(): .catch() is used to handle a rejected Promise. Runs only when reject() is called.

✓ Example:

javascript

Copy code

```
const promise = new Promise((resolve, reject) => {  
  resolve("Success");  
});  
  
promise.then((result) => {  
  console.log(result); // Success  
});
```

- ✓ Task completed successfully
- ✓ Value is available

✓ Example:

javascript

Copy code

```
const promise = new Promise((resolve, reject) => {  
  reject("Error occurred");  
});  
  
promise.catch((error) => {  
  console.log(error); // Error occurred  
});
```

- ✓ Task failed
- ✓ Error is returned

13. What is the JavaScript XML and how helps in building React components.?

Answer:

JSX (JavaScript XML) is a syntax extension for JavaScript used in **React**.

It allows us to write **HTML-like code inside JavaScript**.

👉 JSX is not HTML.

👉 Browsers don't understand JSX directly.

👉 It gets converted into normal JavaScript using tools like **Babel**.

◆ Same example using JSX (Easy to Read)

javascript

📄 Copy code

```
const element = <h1>Hello World</h1>;
```

JSX makes code:

- ✅ Cleaner
- ✅ Easier to read
- ✅ Easier to write

🔑 How JSX Helps in Building React Components

◆ 1. Makes UI Writing Simple

We can design UI using HTML-like syntax.

javascript

📄 Copy code

```
function Welcome() {  
  return <h1>Welcome to React</h1>;  
}
```

This looks just like HTML, so it's easy to understand.

◆ 2. Allows JavaScript Inside HTML

We can use JavaScript inside JSX using { }.

javascript

📄 Copy code

```
function Greeting() {  
  const name = "Shaik";  
  return <h1>Hello {name}</h1>;  
}
```

👉 {name} inserts JavaScript value into UI.

14. What is JSX? Why is it used in React?

Answer:

JSX (JavaScript XML) is a syntax extension for JavaScript used in **React**.

It allows us to write **HTML-like code inside JavaScript**.

👉 JSX is not HTML.

👉 Browsers don't understand JSX directly.

👉 It gets converted into normal JavaScript using tools like **Babel**.

◆ Same example using JSX (Easy to Read)

javascript

📄 Copy code

```
const element = <h1>Hello World</h1>;
```

JSX makes code:

- ✅ Cleaner
- ✅ Easier to read
- ✅ Easier to write

🔗 How JSX Helps in Building React Components

◆ 1. Makes UI Writing Simple

We can design UI using HTML-like syntax.

javascript

📄 Copy code

```
function Welcome() {  
  return <h1>Welcome to React</h1>;  
}
```

This looks just like HTML, so it's easy to understand.

◆ 2. Allows JavaScript Inside HTML

We can use JavaScript inside JSX using { }.

javascript

📄 Copy code

```
function Greeting() {  
  const name = "Shaik";  
  return <h1>Hello {name}</h1>;  
}
```

👉 {name} inserts JavaScript value into UI.

15. What is React and why is it used for building user interfaces?

Answer:

❏ What is React?

React is a **JavaScript library** used for building **User Interfaces (UI)**.

- Developed by **Facebook (Meta)**
- Mainly used to build **Single Page Applications (SPA)**
- Used for creating **interactive and dynamic web applications**

👉 React focuses only on the **View layer** (UI part) of an application.

❏ What is a User Interface (UI)?

User Interface means the part of the application that users:

- 👁 See
- 🖱 Click
- 🖥 Interact with

Example:

- Buttons
- Forms
- Login pages
- Dashboards
- Product lists

React helps us build these parts efficiently.

❏ Why is React Used for Building User Interfaces?

💎 1. Component-Based Architecture

React divides UI into **small reusable components**.

Example:

javascript

 Copy code

```
function Header() {  
  return <h1>Welcome</h1>;  
}
```

Now we can reuse `<Header />` anywhere.

✅ Makes code:

- Reusable
- Organized
- Easy to maintain

(LONG QUESTION AND ANSWERS)

1. Explain Default binding and Implicit binding of “this” keyword along with example program?

Answer: -

Default Binding and Implicit Binding of this in JavaScript

In JavaScript, the value of **this** keyword depends on **how a function is called**, not where it is defined. Two important types of binding are **Default Binding** and **Implicit Binding**.

Default Binding

Definition:

Default binding occurs when a regular function is called **without any object reference**

1.2 this in Regular Functions

```
1 function start() {  
2   console.log(this); // Window { }  
3 }  
4 start();
```

In the above example, **this** refers to the **window** object.

Implicit Binding

Definition:

Implicit binding occurs when a function is called **as a method of an object**. In this case:

- **this** refers to the **object that is calling the function**.

1.1 this in Object Methods

```
1 let car = {  
2   color: "blue",  
3   brand: "Audi",  
4   start: function() {  
5     console.log(this); // Object { color: "blue", brand: "Audi", start: f() }  
6   }  
7 };  
8  
9 car.start();
```

In the above example, **this** refers to the **car** object as it's executing the method **start**.

2.What is Error Handling? Explain the Error handling Mechanisms?

Answer: -

Error Handling in JavaScript is the process of detecting, managing, and responding to runtime errors so that the program does not crash unexpectedly.

JavaScript provides built-in mechanisms to handle errors gracefully and maintain smooth execution.

Types of Errors in JavaScript

1. Syntax Error

Occurs when code violates JavaScript rules.

javascript

 Copy code

```
console.log("Hello"
```

(Missing closing bracket)

2. Runtime Error

Occurs during execution.

javascript

 Copy code

```
let x = y + 5; // y is not defined
```

3. Logical Error

Program runs but gives incorrect output.

Error Handling Mechanisms in JavaScript

try...catch...finally

This is the main error handling mechanism.

Syntax:

javascript

 Copy code

```
try {  
    // Code that may cause error  
}  
catch(error) {  
    // Code to handle error  
}  
finally {  
    // Code that always executes  
}
```

try Block

The try block contains the code that may generate an error.

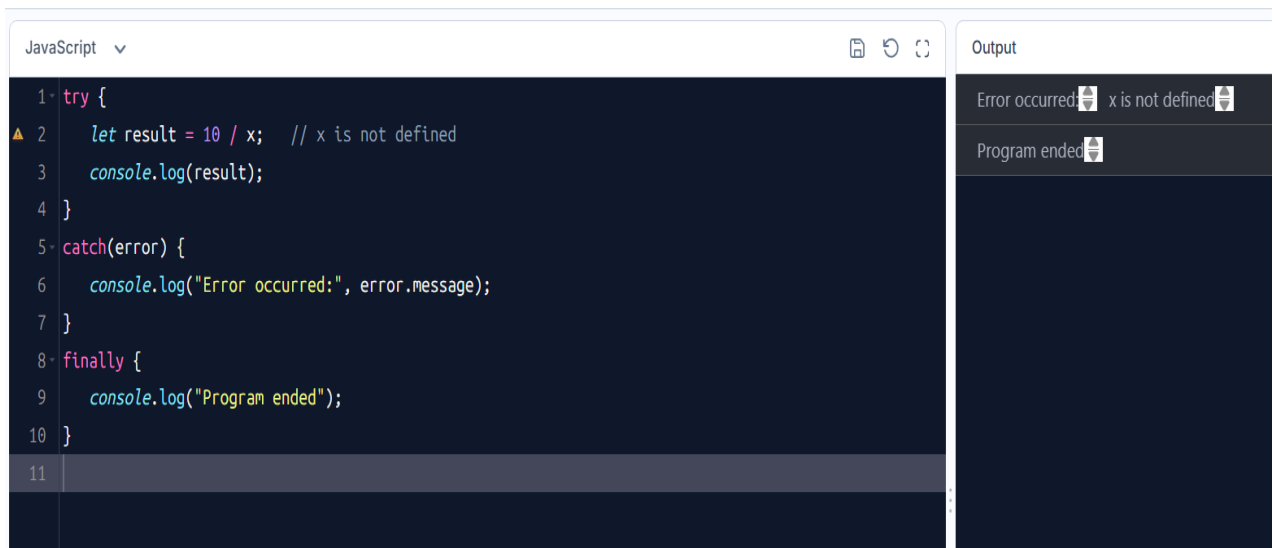
catch Block

The catch block handles the error.

finally Block

The finally block always executes, whether an error occurs or not.

Example: -



The screenshot shows a code editor with the following JavaScript code:

```
1 try {  
2   let result = 10 / x; // x is not defined  
3   console.log(result);  
4 }  
5 catch(error) {  
6   console.log("Error occurred:", error.message);  
7 }  
8 finally {  
9   console.log("Program ended");  
10 }  
11
```

The output panel on the right shows the following messages:

```
Output  
Error occurred: x is not defined  
Program ended
```

3. Explain try, catch, finally block in JavaScript along with example program?

Answer: -

Error Handling in JavaScript is the process of detecting, managing, and responding to runtime errors so that the program does not crash unexpectedly.

JavaScript provides built-in mechanisms to handle errors gracefully and maintain smooth execution.

Types of Errors in JavaScript

1. Syntax Error

Occurs when code violates JavaScript rules.

javascript

 Copy code

```
console.log("Hello"
```

(Missing closing bracket)

2. Runtime Error

Occurs during execution.

javascript

 Copy code

```
let x = y + 5; // y is not defined
```

3. Logical Error

Program runs but gives incorrect output.

Error Handling Mechanisms in JavaScript

try...catch...finally

This is the main error handling mechanism.

Syntax:

javascript

 Copy code

```
try {  
    // Code that may cause error  
}  
catch(error) {  
    // Code to handle error  
}  
finally {  
    // Code that always executes  
}
```

try Block

The try block contains the code that may generate an error.

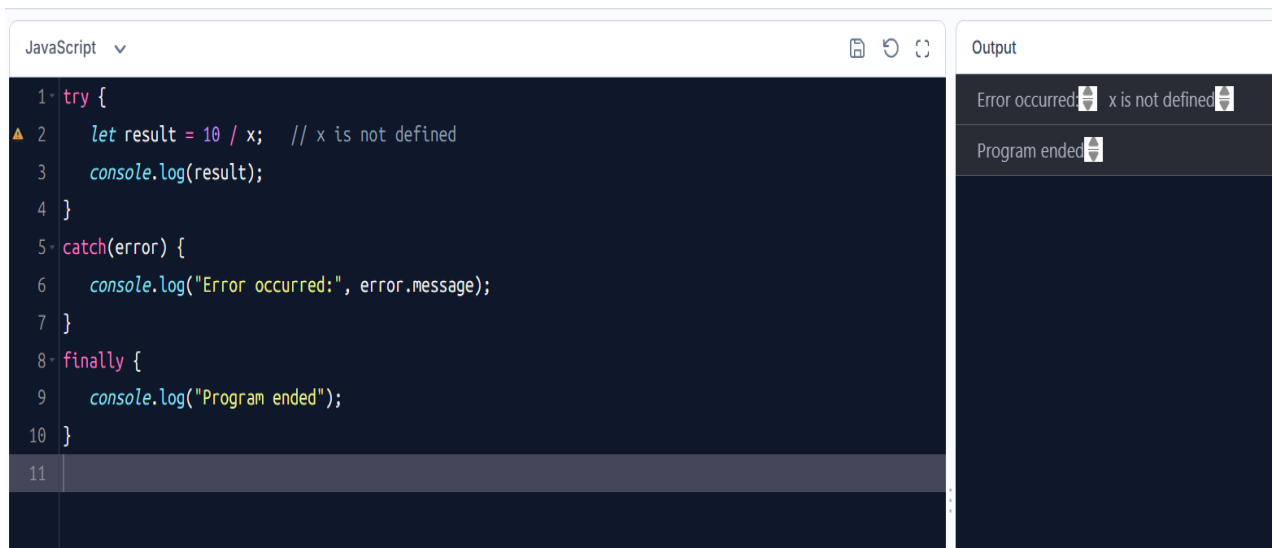
catch Block

The catch block handles the error.

finally Block

The finally block always executes, whether an error occurs or not.

Example: -



The screenshot shows a code editor with a JavaScript file. The code defines a try-catch-finally block. The try block attempts to calculate 10 divided by x, where x is not defined, causing a runtime error. The catch block logs the error message. The finally block logs "Program ended". The output console on the right shows the error message "Error occurred: x is not defined" and "Program ended".

```
1 try {  
2   let result = 10 / x; // x is not defined  
3   console.log(result);  
4 }  
5 catch(error) {  
6   console.log("Error occurred:", error.message);  
7 }  
8 finally {  
9   console.log("Program ended");  
10 }  
11
```

Output

Error occurred: x is not defined

Program ended

4. Explain Classes in JavaScript in detail along with example?

Answer: -

Classes in JavaScript are a blueprint for creating objects. They were introduced in **ECMAScript 6 (ES6)** in 2015.

Classes provide a cleaner and more structured way to create objects and handle inheritance compared to traditional constructor functions.

1.1. Constructor Method

The constructor method is a special method of a class for creating and initializing an object of that class.

Syntax:

```
class MyClass {  
  constructor(property1, property2) {  
    this.property1 = property1;  
    this.property2 = property2;  
  }  
  method1() { ... }  
  method2() { ... }  
}
```

Example: -

```
class Student {  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
  
  greet() {  
    console.log("Hello, my name is " + this.name);  
  }  
}  
  
let s1 = new Student("Rahul", 20);  
console.log(s1)  
console.log(s1.name)  
console.log(s1.greet())
```

Output

```
Object { name: "Rahul", age: 20 }
```

```
Rahul
```

```
Hello, my name is Rahul
```

Inheritance (extends keyword)

One class can inherit properties and methods from another class.

```
1 class Person {
2     constructor(name) {
3         this.name = name;
4     }
5     greet() {
6         console.log("Hello " + this.name);
7     }
8 }
9 class Employee extends Person {
10     constructor(name, salary) {
11         super(name); // Call parent constructor
12         this.salary = salary;
13     }
14     showSalary() {
15         console.log("Salary: " + this.salary);
16     }
17 }
18 let emp1 = new Employee("Amit", 50000);
19 emp1.greet();
20 emp1.showSalary();
21
```

Output

Hello Amit

Salary: 50000

5. Explain any ten Array Methods in JS along with example?

✓ 1. `push()`

► Definition:

Adds one or more elements to the **end** of an array.

► Syntax:

javascript

 Copy code

```
array.push(element1, element2, ...)
```

► Example:

javascript

 Copy code

```
let arr = [1, 2, 3];  
arr.push(4);  
console.log(arr); // [1, 2, 3, 4]
```



✓ 2. `pop()`

► Definition:

Removes the **last element** from an array.

► Syntax:

javascript

 Copy code

```
array.pop()
```

► Example:

javascript

 Copy code

```
let arr = [1, 2, 3];  
arr.pop();  
console.log(arr); // [1, 2]
```



✓ 3. `shift()`

► Definition:

Removes the **first element** from an array.

► Syntax:

javascript

 Copy code

```
array.shift()
```

► Example:

javascript

 Copy code

```
let arr = [10, 20, 30];  
arr.shift();  
console.log(arr); // [20, 30]
```



✓ 4. `unshift()`

► Definition:

Adds one or more elements to the **beginning** of an array.

► Syntax:

javascript

 Copy code

```
array.unshift(element1, element2, ...)
```

► Example:

javascript

 Copy code

```
let arr = [2, 3];  
arr.unshift(1);  
console.log(arr); // [1, 2, 3]
```



✓ 5. `map()`

► Definition:

Creates a **new array** by applying a function to each element.

► Syntax:

javascript

 Copy code

```
array.map(function(element, index, array))
```

► Example:

javascript

 Copy code

```
let arr = [1, 2, 3];  
let doubled = arr.map(num => num * 2);  
console.log(doubled); // [2, 4, 6]
```



✓ 6. `filter()`

► Definition:

Creates a **new array** with elements that pass a condition.

► Syntax:

javascript

 Copy code

```
array.filter(function(element))
```

► Example:

javascript

 Copy code

```
let arr = [1, 2, 3, 4];  
let even = arr.filter(num => num % 2 === 0);  
console.log(even); // [2, 4]
```



✓ 7. `reduce()`

► Definition:

Reduces array to a **single value**.

► Syntax:

javascript

 Copy code

```
array.reduce(function(accumulator, currentValue), initialValue)
```

► Example:

javascript

 Copy code

```
let arr = [1, 2, 3, 4];  
let sum = arr.reduce((total, num) => total + num, 0);  
console.log(sum); // 10
```



✓ 8. `forEach()`

► Definition:

Executes a function for each array element.

► Syntax:

javascript

 Copy code

```
array.forEach(function(element, index))
```

► Example:

javascript

 Copy code

```
let arr = [1, 2, 3];  
arr.forEach(num => console.log(num));
```



✓ 9. slice()

► Definition:

Returns a **new array** by extracting a portion of the array.

► Syntax:

javascript

Copy code

```
array.slice(start, end)
```

► Example:

javascript

Copy code

```
let arr = [10, 20, 30, 40];
let part = arr.slice(1, 3);
console.log(part); // [20, 30]
```



✓ 10. length

► Definition:

Returns the **number of elements** in an array.

It can also be used to **modify the size** of an array.

► Syntax:

javascript

Copy code

```
array.length
```

► Example 1: Getting length

javascript

Copy code

```
let arr = [10, 20, 30, 40];
console.log(arr.length); // 4
```



6.Explain any ten String Methods in JS along with example?

✓ 1. `length`

► Definition:

Returns the **number of characters** in a string.

► Syntax:

```
javascript
```

[Copy code](#)

```
string.length
```

► Example:

```
javascript
```

[Copy code](#)

```
let str = "JavaScript";  
console.log(str.length); // 10
```



✓ 2. `toUpperCase()`

► Definition:

Converts all characters in a string to **uppercase**.

► Syntax:

```
javascript
```

[Copy code](#)

```
string.toUpperCase()
```

► Example:

```
javascript
```

[Copy code](#)

```
let str = "hello";  
console.log(str.toUpperCase()); // HELLO
```

✓ 3. toLowerCase()

► Definition:

Converts all characters in a string to **lowercase**.

► Syntax:

javascript

 Copy code

```
string.toLowerCase()
```

► Example:

javascript

 Copy code

```
let str = "HELLO";  
console.log(str.toLowerCase()); // hello
```

✓ 4. trim()

► Definition:

Removes whitespace from both ends of a string.

► Syntax:

javascript

 Copy code

```
string.trim()
```

► Example:

javascript

 Copy code

```
let str = " Hello World ";  
console.log(str.trim()); // "Hello World"
```



✓ 5. `charAt()`

► Definition:

Returns the character at a specified index.

► Syntax:

javascript

Copy code

```
string.charAt(index)
```

► Example:

javascript

Copy code

```
let str = "Java";  
console.log(str.charAt(2)); // v
```



✓ 6. `indexOf()`

► Definition:

Returns the position of the **first occurrence** of a specified value.

► Syntax:

javascript

Copy code

```
string.indexOf(searchValue)
```

► Example:

javascript

Copy code

```
let str = "Hello World";  
console.log(str.indexOf("World")); // 6
```



✓ 7. `lastIndexOf()`

► Definition:

Returns the position of the **last occurrence** of a specified value.

► Syntax:

javascript

 Copy code

```
string.lastIndexOf(searchValue)
```

► Example:

javascript

 Copy code

```
let str = "hello hello";  
console.log(str.lastIndexOf("hello")); // 6
```

✓ 8. `slice()`

► Definition:

Extracts a part of a string and returns a new string.

► Syntax:

javascript

 Copy code

```
string.slice(start, end)
```

► Example:

javascript

 Copy code

```
let str = "JavaScript";  
console.log(str.slice(0, 4)); // Java
```

✓ 9. includes()

► Definition:

Checks whether a string contains a specified value.

► Syntax:

javascript

 Copy code

```
string.includes(searchValue)
```

► Example:

javascript

 Copy code

```
let str = "I love JavaScript";  
console.log(str.includes("Java")); // ↓
```

✓ 10. concat()

► Definition:

Joins two or more strings.

► Syntax:

javascript

 Copy code

```
string.concat(string1, string2, ...)
```

► Example:

javascript

 Copy code

```
let str1 = "Hello";  
let str2 = "World";  
console.log(str1.concat(" ", str2)); // ↓ Hello World
```


7. Explain Callback Function along with example program?

A **callback function** is a function that is **passed as an argument to another function** and is executed **after the completion of that function**.

Callbacks are very common in **JavaScript**, especially for:

- Asynchronous operations (like API calls)
- Event handling (button clicks)
- Timers (setTimeout)

3 Syntax Structure

javascript

 Copy code

```
function firstFunction(callback) {  
    // some task  
    callback(); // calling the callback function  
}  
  
function secondFunction() {  
    console.log("This is callback function");  
}  
  
firstFunction(secondFunction);
```



Example: -

JavaScript

```
1 function greet(name, callback) {  
2     console.log("Hello " + name);  
3     callback();  
4 }  
5 function sayBye() {  
6     console.log("Goodbye!");  
7 }  
8 greet("Rahul", sayBye);  
9
```

Output

Hello Rahul

Goodbye!

5 Example with setTimeout (Asynchronous Callback)

javascript

 Copy code

```
function displayMessage() {  
    console.log("Message displayed after 2 seconds");  
}  
  
setTimeout(displayMessage, 2000);
```

Explanation:

- `setTimeout` takes a callback function
- After 2 seconds, `displayMessage` is executed

8. Explain Callback Hell in JavaScript in detail?

Answer:

Callback Hell is a situation where multiple asynchronous operations are nested inside one another using callbacks, resulting in deeply indented and hard-to-read code.

It is also called:

- ♦ Pyramid of Doom
- ♦ Nested Callback Problem

Why Does Callback Hell Occur?

JavaScript is single-threaded. (One task at a time)

To handle time-consuming tasks, it uses **asynchronous programming**.

When tasks depend on each other, developers often write:

- Callback inside callback
- Inside another callback
- And so on

This creates a pyramid structure.

Example of Callback Hell

JavaScript ▾

```
1 setTimeout(() => {  
2   console.log("Step 1: Login");  
3  
4   setTimeout(() => {  
5     console.log("Step 2: Fetch User Data");  
6  
7     setTimeout(() => {  
8       console.log("Step 3: Fetch Orders");  
9  
10      setTimeout(() => {  
11        console.log("Step 4: Payment Done");  
12      }, 1000);  
13    }, 1000);  
14  }, 1000);  
15  
16  }, 1000);  
17  
18 }, 1000);  
19
```

Output

Step 1: Login 

Step 2: Fetch User Data 

Step 3: Fetch Orders 

Step 4: Payment Done 

Problems of Callback Hell

1. Poor Readability

Code becomes confusing due to deep nesting.

2. Difficult Debugging

Finding where an error occurs is harder.

3. Complex Error Handling

Each callback needs separate error handling.

4. Hard to Maintain

Updating or modifying nested code becomes difficult.

How to Avoid Callback Hell

1. Use Promises

```
JavaScript  ▾  
1  const myPromise = new Promise((resolved, rejected) => {  
2    let marks = 70;  
3    setTimeout(() => {  
4      if (marks >= 50) {  
5        resolved("Student Passed ✅");  
6      } else {  
7        rejected("Student Failed ❌");  
8      }  
9    }, 1000);  
10 })  
11 .then((result) => {  
12   console.log(result);  
13 })  
14 .catch((error) => {  
15   console.log(error);  
16 });  
17
```

2. Use Async/Await

```
JavaScript  ▾  
1  const myPromise = new Promise((resolved, rejected) => {  
2    let marks = 50;  
3    setTimeout(() => {  
4      if (marks >= 50) {  
5        resolved("Student Passed ✅");  
6      } else {  
7        rejected("Student Failed ❌");  
8      }  
9    }, 1000);  
10  });  
11  async function checkResult() {  
12    try {  
13      const result = await myPromise;  
14      console.log(result);  
15    } catch (error) {}  
16    console.log(error);  
17  }  
18  console.log("hello")  
19  }  
20  checkResult();  
21
```

Output

Student Passed ✅
hello

9.Explain the concept of “Pyramid of Doom” in detail along with example program?

Answer:

Callback Hell is a situation where multiple asynchronous operations are nested inside one another using callbacks, resulting in deeply indented and hard-to-read code.

It is also called:

- ◆ Pyramid of Doom
 - ◆ Nested Callback Problem
-

Why Does Callback Hell Occur?

JavaScript is single-threaded. (One task at a time)

To handle time-consuming tasks, it uses **asynchronous programming**.

When tasks depend on each other, developers often write:

- Callback inside callback
- Inside another callback
- And so on

This creates a pyramid structure.

Example of Callback Hell

JavaScript ▾

```
1 setTimeout(() => {  
2   console.log("Step 1: Login");  
3  
4   setTimeout(() => {  
5     console.log("Step 2: Fetch User Data");  
6  
7     setTimeout(() => {  
8       console.log("Step 3: Fetch Orders");  
9  
10      setTimeout(() => {  
11        console.log("Step 4: Payment Done");  
12      }, 1000);  
13    }, 1000);  
14  }, 1000);  
15  
16  }, 1000);  
17  
18 }, 1000);  
19
```

Output

Step 1: Login 

Step 2: Fetch User Data 

Step 3: Fetch Orders 

Step 4: Payment Done 

Problems of Callback Hell

1. Poor Readability

Code becomes confusing due to deep nesting.

2. Difficult Debugging

Finding where an error occurs is harder.

3. Complex Error Handling

Each callback needs separate error handling.

4. Hard to Maintain

Updating or modifying nested code becomes difficult.

How to Avoid Callback Hell

Use Promises

```
JavaScript ▾  
1 const myPromise = new Promise((resolved, rejected) => {  
2   let marks = 70;  
3   setTimeout(() => {  
4     if (marks >= 50) {  
5       resolved("Student Passed ✅");  
6     } else {  
7       rejected("Student Failed ❌");  
8     }  
9   }, 1000);  
10 })  
11 .then((result) => {  
12   console.log(result);  
13 })  
14 .catch((error) => {  
15   console.log(error);  
16 });  
17
```

Use Async/Await

JavaScript ▾

```
1 const myPromise = new Promise((resolved, rejected) => {
2   let marks = 50;
3   setTimeout(() => {
4     if (marks >= 50) {
5       resolved("Student Passed ✅");
6     } else {
7       rejected("Student Failed ❌");
8     }
9   }, 1000);
10 });
11 async function checkResult() {
12   try {
13     const result = await myPromise;
14     console.log(result);
15   } catch (error) {}
16   console.log(error);
17 }
18 console.log("hello")
19 }
20 checkResult();
21
```

Output

Student Passed ✅

hello

10.Explain timer functions along with example program?

Answer:

Timer Functions in JavaScript

Timer functions are used to execute code after a specific delay or repeatedly at fixed time intervals. JavaScript provides two main timer functions:

1. **setTimeout()**
2. **setInterval()**

1) setTimeout() Function

► Definition:

setTimeout() is used to execute a function **once** after a specified time delay (in milliseconds).

► Syntax:

javascript

 Copy code

```
setTimeout(function, delay, parameter1, parameter2, ...);
```

- **function** → The function to execute
- **delay** → Time in milliseconds (1000 ms = 1 second)

► Example Program:

javascript

 Copy code

```
function greet() {  
    console.log("Hello after 3 seconds!");  
}  
  
setTimeout(greet, 3000);
```

► Output:

After 3 seconds, it prints:

nginx

 Copy code

```
Hello after 3 seconds!
```

2) setInterval() Function

► Definition:

setInterval() is used to execute a function **repeatedly** at specified time intervals.

► Syntax:

javascript

 Copy code

```
setInterval(function, interval);
```

- `function` → The function to execute
- `interval` → Time in milliseconds

► Example Program:

javascript

 Copy code

```
function showTime() {  
    console.log("Running every 2 seconds");  
}  
  
setInterval(showTime, 2000);
```

► Output:

It prints every 2 seconds continuously:

sql

 Copy code

```
Running every 2 seconds  
Running every 2 seconds  
Running every 2 seconds  
...
```



Note: Since the setInterval() method executes the function infinitely hence there is a method called as clearInterval() to stop the execution of the setInterval().

Example using clearInterval() to stop the execution of the setInterval().

```
JavaScript
1 let count = 0;
2
3 let interval = setInterval(() => {
4     count++;
5     console.log("Count:", count);
6
7     if (count === 5) {
8         clearInterval(interval);
9     }
10 }, 1000);
```

Output

Count	1
Count	2
Count	3
Count	4
Count	5

11. Explain setTimeout, setInterval functions in detail along with example?

Answer:

1) setTimeout() Function

► Definition:

setTimeout() is used to execute a function **once** after a specified time delay (in milliseconds).

► Syntax:

javascript

 Copy code

```
setTimeout(function, delay, parameter1, parameter2, ...);
```

- **function** → The function to execute
- **delay** → Time in milliseconds (1000 ms = 1 second)

► Example Program:

javascript

 Copy code

```
function greet() {
    console.log("Hello after 3 seconds!");
}

setTimeout(greet, 3000);
```

► Output:

After 3 seconds, it prints:

nginx

 Copy code

```
Hello after 3 seconds!
```

2) setInterval() Function

► Definition:

setInterval() is used to execute a function **repeatedly** at specified time intervals.

► Syntax:

javascript

 Copy code

```
setInterval(function, interval);
```

- `function` → The function to execute
- `interval` → Time in milliseconds

► Example Program:

javascript

 Copy code

```
function showTime() {  
    console.log("Running every 2 seconds");  
}  
  
setInterval(showTime, 2000);
```

► Output:

It prints every 2 seconds continuously:

sql

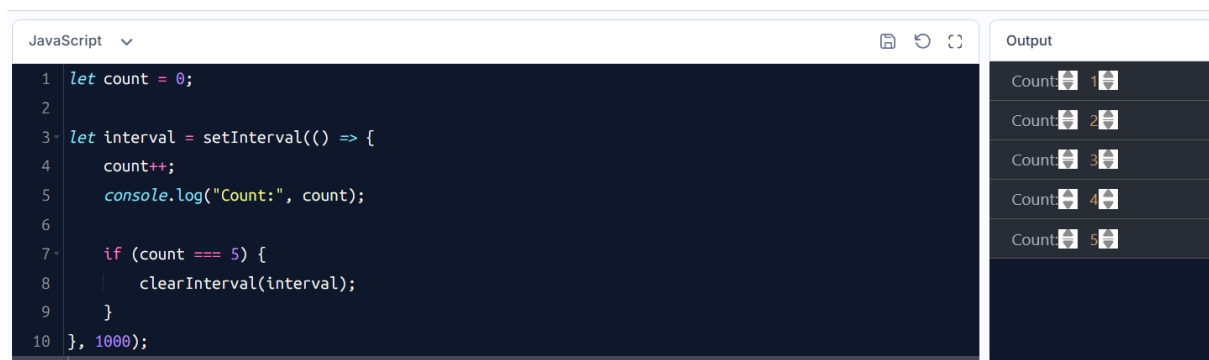
 Copy code

```
Running every 2 seconds  
Running every 2 seconds  
Running every 2 seconds  
...
```



Note: Since the setInterval() method executes the function infinitely hence there is a method called as clearInterval() to stop the execution of the setInterval().

Example using `clearInterval()` to stop the execution of the `setInterval()`.



```
1 let count = 0;
2
3 let interval = setInterval(() => {
4   count++;
5   console.log("Count:", count);
6
7   if (count === 5) {
8     clearInterval(interval);
9   }
10 }, 1000);
```

Output

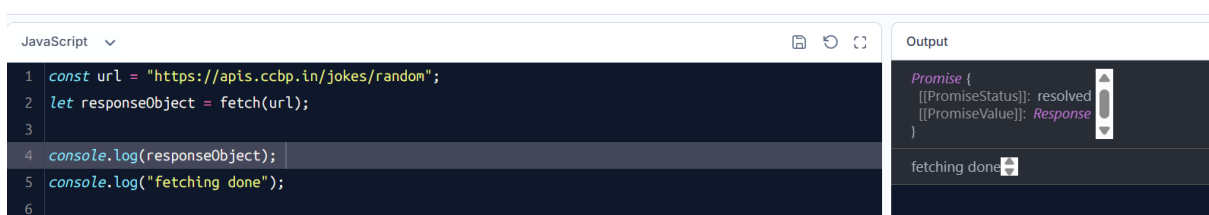
```
Count 1
Count 2
Count 3
Count 4
Count 5
```

12. Explain Promise function and its structure? How do we create promise?

Answer:

JS Promises

1. Promise is a way to handle **Asynchronous** operations.
2. A promise is an object that represents a result of operation that will be returned at some point in the future.



```
1 const url = "https://apis.ccbp.in/jokes/random";
2 let responseObject = fetch(url);
3
4 console.log(responseObject);
5 console.log("fetching done");
6
```

Output

```
Promise {
  [[PromiseStatus]]: resolved
  [[PromiseValue]]: Response
}
fetching done
```

States of a Promise

A Promise has three states:

1. **Pending** – Initial state (operation is not completed)
2. **Fulfilled (Resolved)** – Operation completed successfully
3. **Rejected** – Operation failed

3 Structure of a Promise

The basic structure of a Promise:

```
javascript Copy code  
  
let promise = new Promise(function(resolve, reject) {  
  // asynchronous task  
  
  if (success) {  
    resolve("Success message");  
  } else {  
    reject("Error message");  
  }  
});
```

Explanation:

- `new Promise()` → Creates a Promise object
- `resolve()` → Called when operation is successful
- `reject()` → Called when operation fails
- `.then()` → Handles success
- `.catch()` → Handles error



Example: -

```
JavaScript ▼  
  
1 const myPromise = new Promise((resolved, rejected) => {  
2   let marks = 70;  
3   setTimeout(() => {  
4     if (marks >= 50) {  
5       resolved("Student Passed ✅");  
6     } else {  
7       rejected("Student Failed ❌");  
8     }  
9   }, 1000);  
10 })  
11 .then((result) => {  
12   console.log(result);  
13 })  
14 .catch((error) => {  
15   console.log(error);  
16 });  
17
```

13. Explain the types of components in React in detail?

Answer:

There are mainly **two types of components in React**:

1. Functional Components
2. Class Components

1) Functional Components

Functional components are simple JavaScript functions that return JSX.

◆ Definition

A functional component is a JavaScript function that accepts **props** as an argument and returns JSX.

◆ Syntax

javascript

 Copy code

```
function Welcome(props) {  
  return <h1>Hello, {props.name}</h1>;  
}
```

OR (Arrow Function)

javascript

 Copy code

```
const Welcome = (props) => {  
  return <h1>Hello, {props.name}</h1>;  
};
```

◆ Features

- Simple and easy to write
- Use **Hooks** to manage state and lifecycle methods
- Preferred in modern React
- Better performance and readability

2) Class Components

Class components are ES6 classes that extend `React.Component`.

◆ Definition

A class component is a JavaScript class that extends `React.Component` and contains a `render()` method that returns JSX.

◆ Syntax

javascript

Copy code

```
import React, { Component } from "react";

class Welcome extends Component {
  render() {
    return <h1>Hello, {this.props.name}</h1>;
  }
}
```

◆ Features

- Uses `this`
- Contains a `render()` method
- Can manage state using `this.state`
- Lifecycle methods like:
 - `componentDidMount()`
 - `componentDidUpdate()`
 - `componentWillUnmount()`

14.Explain the concept of React in detail?

Answer:

Introduction

React is a popular JavaScript library used for building user interfaces, especially for single-page applications (SPAs). It was developed and maintained by **Facebook** (now **Meta Platforms**). React allows developers to create reusable UI components and efficiently update the user interface when data changes.

1) What is React?

React is:

- A **JavaScript library** (not a framework)
- Used mainly for building **front-end UI**
- Based on a **component-based architecture**

- Efficient due to its **Virtual DOM**

2) Key Concepts of React

❑ Component-Based Architecture

React applications are built using **components**. A component is a reusable piece of code that returns JSX.

Example:

javascript

 Copy code

```
function Welcome() {  
  return <h1>Hello, React!</h1>;  
}
```

Components make the application:

- Modular
- Reusable
- Easy to maintain

JSX (JavaScript XML)

JSX allows writing HTML-like code inside JavaScript.

Example

🗨 Ask ChatGPT

javascript

 Copy code

```
const element = <h1>Hello World</h1>;
```

JSX:

- Makes code readable
- Combines UI and logic
- Gets converted to JavaScript by Babel

State and Props

Props:

- Used to pass data from one component to another.

- Read-only.

State:

- Stores data that changes over time.
- Managed inside the component.

15. Compare functional components and class components in React with examples.

Answer:

Basis	Functional Components	Class Components
Definition	JavaScript functions that return JSX.	ES6 classes that extend React.Component.
Syntax	Simple function syntax.	Uses class keyword and render() method.
State Management	Uses Hooks like useState().	Uses this.state and this.setState().
Lifecycle Methods	Uses Hooks like useEffect().	Has lifecycle methods like componentDidMount(), componentDidUpdate().
Code Length	Short and simple.	More verbose and lengthy.
Performance	Slightly better (lighter syntax).	Slightly heavier compared to functional components.
this Keyword	Does not use this.	Uses this keyword frequently.
Readability	Easier to read and maintain.	More complex for beginners.